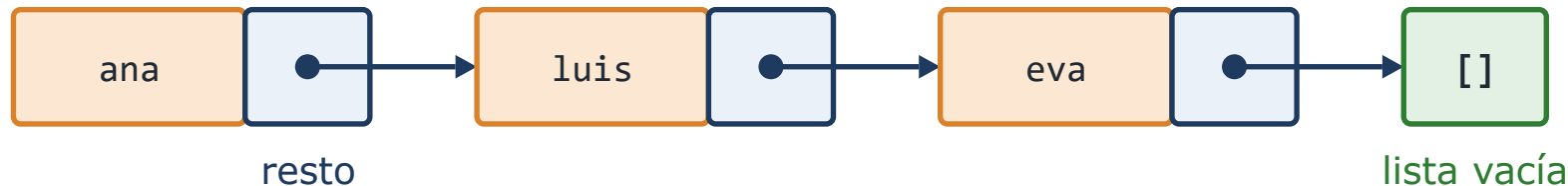


Capítulo 7 — Listas

Una lista es un término

`[ana, luis, eva]`

primer elemento

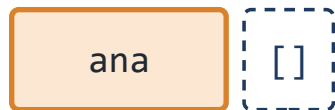


```
[ana, luis, eva] = [ana|[luis|[eva|[]]]]
```

los corchetes evitan escribir la anidación

Primer elemento y resto

lista



Primero

Resto

ana

[luis, eva]

ana

[]

❌ no tiene primer elemento

La descomposición, en Prolog

```
?- [ana, luis, eva] = [Primero|Resto].
```

```
Primero = ana,
```

```
Resto = [luis, eva].
```

```
?- [ana] = [Primero|Resto].
```

```
Primero = ana,
```

```
Resto = [].
```

```
?- [] = [__].
```

```
false.
```

Más de un elemento al comienzo

```
?- [Primero, Segundo|Resto] = [ana, luis, eva, sofia].
```

```
Primero = ana,
```

```
Segundo = luis,
```

```
Resto = [eva, sofia].
```

Dos maneras de ser una lista

una lista es...

`[]`

la lista vacía

`[X|Resto]`

un primer elemento
y una lista

o

`p([]) :- ...`

`p([X|Resto]) :- ..., p(Resto).`

el programa reproduce la estructura que recorre

esta_en/2

```
esta_en(X, [X|_]).  
esta_en(X, [_|Resto]) :-  
    esta_en(X, Resto).
```

Buscar a luis

`esta_en(luis, [ana, luis, eva])`

una lista más corta

`esta_en(luis, [luis, eva])`

- ❌ cláusula 1: luis no es ana
- 2 cláusula 2: sigue con el resto
- ✅ cláusula 1: luis es el primero

`esta_en(X, [X|_]).`

la misma variable dos veces: la unificación impone la igualdad

Verificar y enumerar

```
?- esta_en(X, [ana, luis, eva]).
```

```
X = ana ;
```

```
X = luis ;
```

```
X = eva ;
```

```
false.
```

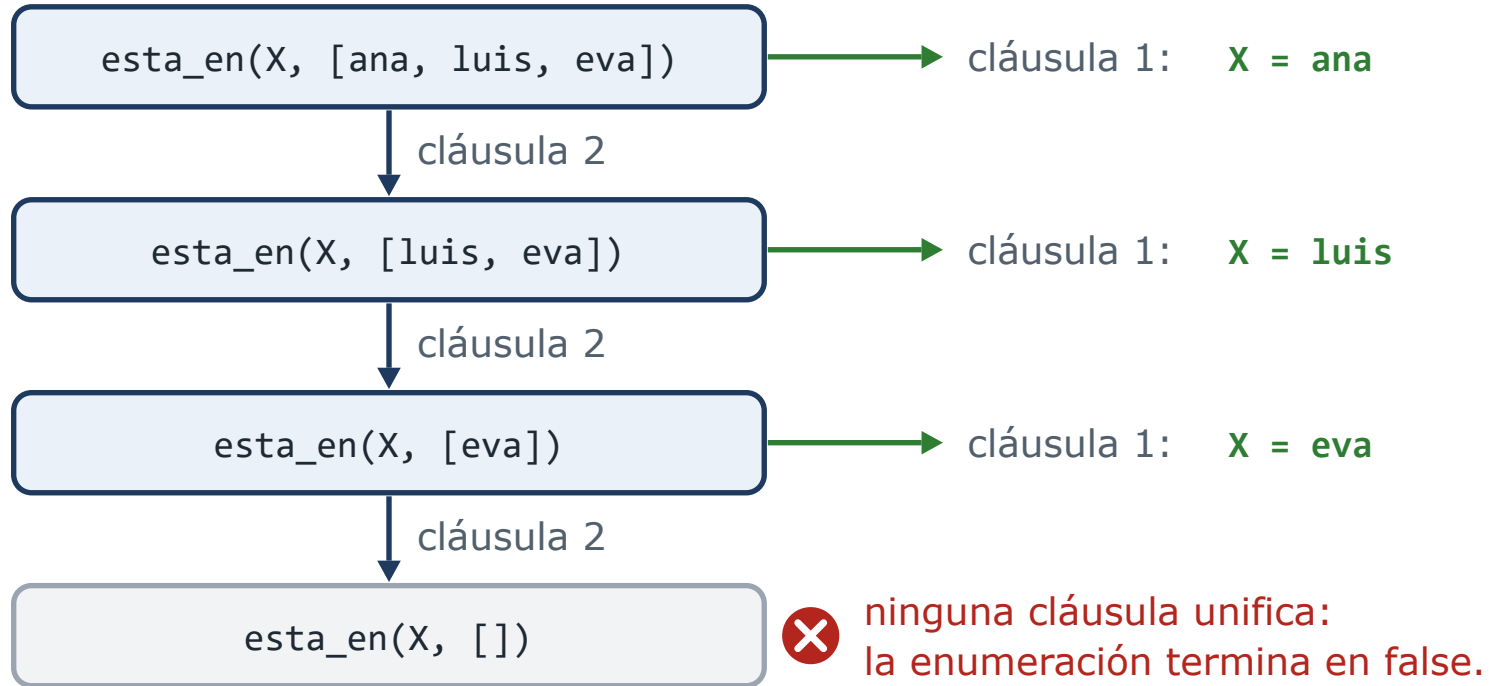
```
?- esta_en(ana, [ana, ana]).
```

```
true ;
```

```
true ;
```

```
false.
```

Por qué termina en false



Plantilla 9 — Recorrer una lista

```
p([]) :-  
    caso_de_la_lista_vacia.
```

caso base

la lista vacía

```
p([Primero|Resto]) :-  
    procesar(Primero),  
    p(Resto).
```

caso recursivo

procesa el primero
y sigue con el resto

A veces el caso base es un hecho; otras veces
no hace falta escribirlo.

Plantilla 10 — Buscar uno que cumple

```
p([Primer_|_]) :-  
    cumple(Primeros).
```

éxito sin recursión

en cuanto uno cumple

```
p([_|Resto]) :-  
    p(Resto).
```

caso recursivo

sigue con el resto

Sobre [] no hay cláusula aplicable: ninguno cumplió.
Una respuesta por cada elemento que cumple.

todos_estan/2

```
todos_estan([], _).  
todos_estan([X|Resto], L) :-  
    esta_en(X, L),  
    todos_estan(Resto, L).  
  
?- todos_estan([ana, eva], [ana, luis, eva]).  
true ;  
false.
```

Plantilla 11 — Todos cumplen

```
p([]).
```

caso base

sobre [], «todos cumplen»
es cierto

```
p([Primero|Resto]) :-  
    cumple(Primero),  
    p(Resto).
```

caso recursivo

prueba el primero
y sigue con el resto

Fracasa en cuanto un elemento no cumple.
No produce resultado: se cumple o no.

Tres siluetas

9 · recorrer

procesa cada
elemento

sobre []:
el caso base

en el capítulo:
largo/2

10 · buscar

alguno cumple

sobre []:
fracasa

en el capítulo:
esta_en/2

11 · todos

todos cumplen

sobre []:
tiene éxito

en el capítulo:
todos_estan/2

largo/2

```
largo([], 0).  
largo([_|Resto], N) :-  
    largo(Resto, Faltan),  
    N is Faltan + 1.
```

El largo, al retorno

las llamadas bajan

`largo([ana, luis, eva], N)`

`largo([luis, eva], F1)`

`largo([eva], F2)`

`largo([], F3)`

al retorno, + 1

$N = 2 + 1 = 3$

$F1 = 1 + 1 = 2$

$F2 = 0 + 1 = 1$

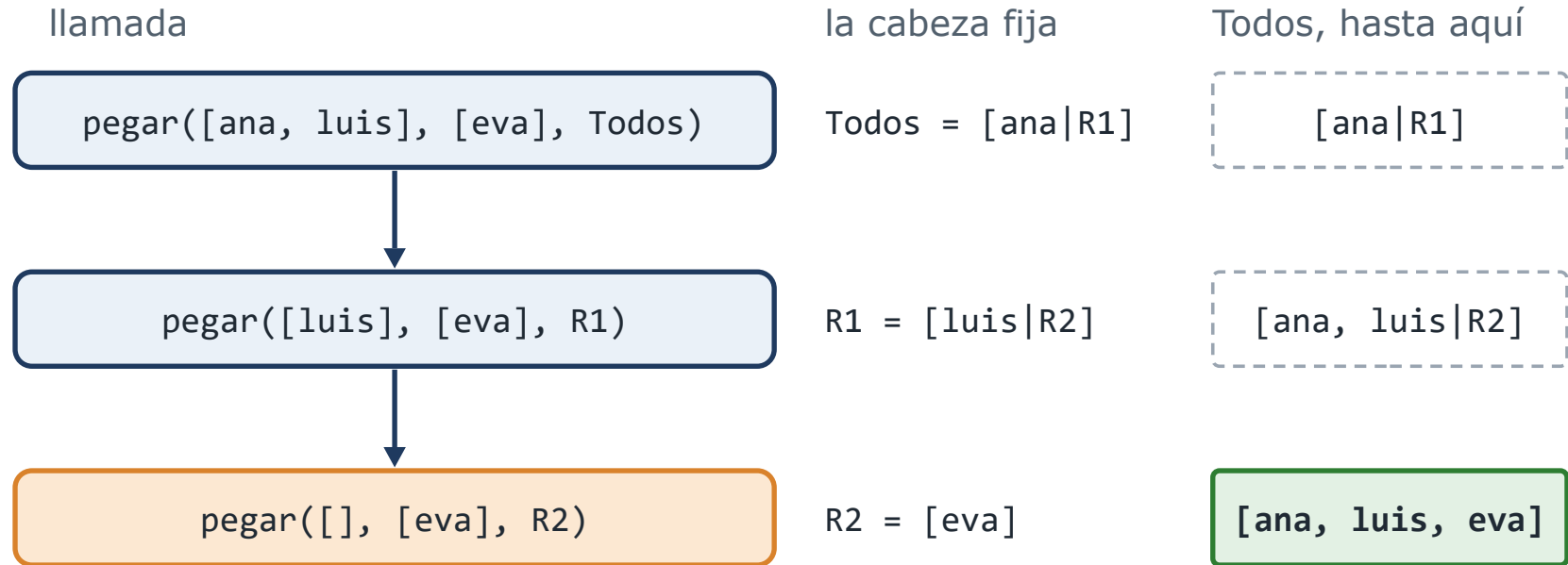
$F3 = 0$

caso base

pegar/3

```
%!  pegar(+A, ?B, -C) is det.  
%!  pegar(?A, ?B, +C) is nondet.  
%  
%  C es la lista A seguida de la lista B.  
pegar([], B, B).  
pegar([X|RestoA], B, [X|RestoC]) :-  
    pegar(RestoA, B, RestoC).  
  
?- pegar([ana, luis], [eva], Todos).  
Todos = [ana, luis, eva].
```

Construcción en la cabeza



lista incompleta en cada paso; el caso base la cierra

Plantilla 12 – Construir una lista

```
p([], CasoBase).
```

caso base

qué resultado
corresponde a []

```
p([X|Resto], [Y|RestoNuevo]) :-  
    relacionar(X, Y),  
    p(Resto, RestoNuevo).
```

caso recursivo

el resultado va en la cabeza:
cada llamada aporta
el primer elemento

Cambian el caso base y la relación entre cada elemento
y el que ocupa su lugar en el resultado.

Una relación, varios sentidos

```
?- pegar(A, B, [ana, luis, eva]).
```

```
A = [],
```

```
B = [ana, luis, eva] ;
```

```
A = [ana],
```

```
B = [luis, eva] ;
```

```
A = [ana, luis],
```

```
B = [eva] ;
```

```
A = [ana, luis, eva],
```

```
B = [] ;
```

```
false.
```

Las cuatro particiones



A = []

B = [ana, luis, eva]



A = [ana]

B = [luis, eva]



A = [ana, luis]

B = [eva]



A = [ana, luis, eva]

B = []

ultimo/2

```
ultimo([X], X).  
ultimo([_|Resto], X) :-  
    ultimo(Resto, X).  
  
?- ultimo([ana, luis, eva], U).  
U = eva ;  
false.
```

ultimo/2, en sentido inverso

```
?- ultimo(L, eva).
```

```
L = [eva] ;
```

```
L = [_ , eva] ;
```

```
L = [_ , _ , eva] ;
```

```
...
```

El límite de los varios sentidos

predicado	termina si...	sin fin si...
pegar/3	la 1. ^a o la 3. ^a lista está completa	la 1. ^a y la 3. ^a están sin determinar
esta_en/2	la lista está completa	la lista está sin determinar
ultimo/2	la lista está completa	la lista está sin determinar

antes de un sentido nuevo: ¿qué recorre, y es finito?

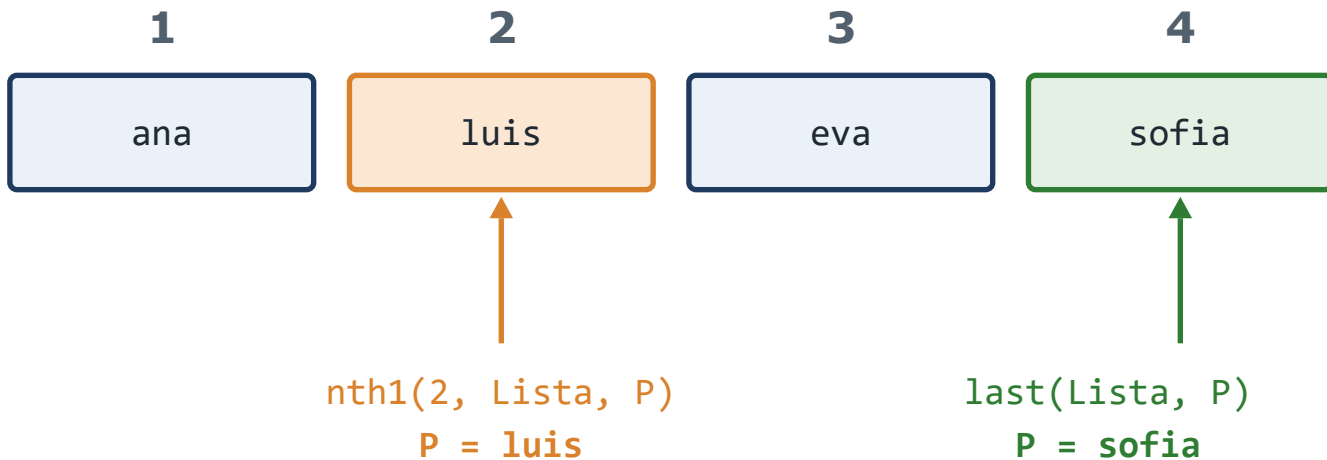
Predicados predefinidos

predefinido	significa	equivale a
<code>member(X, L)</code>	X es un elemento de L	<code>esta_en/2</code>
<code>length(L, N)</code>	L tiene N elementos	<code>largo/2</code>
<code>append(A, B, C)</code>	C es A seguida de B	<code>pegar/3</code>
<code>reverse(L, R)</code>	R es L en orden inverso	—
<code>last(L, X)</code>	X es el último	<code>ultimo/2</code>
<code>nth1(N, L, X)</code>	X está en la posición N	—

Los invitados

```
invitados([ana, luis, eva, sofia]).
```

posición



las posiciones se cuentan desde 1

member, length y nth1

```
esta_invitado(P) :-  
    invitados(Lista),  
    member(P, Lista).
```

```
cuantos(N) :-  
    invitados(Lista),  
    length(Lista, N).
```

```
en_el_puesto(N, P) :-  
    invitados(Lista),  
    nth1(N, Lista, P).
```

```
?- esta_invitado(Quien).
```

```
Quien = ana ;
```

```
Quien = luis ;
```

```
Quien = eva ;
```

```
Quien = sofia.
```

```
?- cuantos(N).
```

```
N = 4.
```

```
?- en_el_puesto(2, Quien).
```

```
Quien = luis.
```

nth1/3, en sentido inverso

```
?- en_el_puesto(N, eva).
```

```
N = 3 ;
```

```
false.
```

last y reverse

```
ultimo_en_llegar(P) :-  
    invitados(Lista),  
    last(Lista, P).
```

```
orden_de_salida(L) :-  
    invitados(Lista),  
    reverse(Lista, L).
```

```
?- ultimo_en_llegar(P).  
P = sofia.
```

```
?- orden_de_salida(L).  
L = [sofia, eva, luis, ana].
```

append, un invitado más

```
con_uno_mas(P, L) :-  
    invitados(Lista),  
    append(Lista, [P], L).
```

```
?- con_uno_mas(pedro, L).  
L = [ana, luis, eva, sofia, pedro].
```

Resumen

`[]` la lista vacía: una constante

`[X|Resto]` primer elemento y resto, que es una lista

`[A, B|Resto]` los dos primeros elementos y el resto

`9 · 10 · 11` recorrer · buscar uno · todos cumplen

`12` construir el resultado en la cabeza

`pegar/3` una relación: concatena y particiona

`append/3 ...` los predefinidos que usa el resto del curso

Créditos

- Ilustraciones: dibujadas para el curso, licencia MIT.
- Ejemplos: ejemplos/capitulo-07 del curso.